

FULL STACK DEVELOPMENT

FASE 4 | AULA 06 -

REDUX TOOLKIT

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON.....	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	12
REFERÊNCIAS	13

EMANDA

O QUE VEM POR AÍ?

Nesta aula, exploraremos o Redux Toolkit, uma poderosa biblioteca para gerenciamento de estado em aplicações React. Vamos aprender o que é Redux Toolkit, seus principais recursos, vantagens e quando usá-lo. No hands-on, faremos uma implementação prática de Redux Toolkit em uma aplicação React, cobrindo desde a configuração inicial até a criação e gerenciamento de slices de estado.



HANDS ON

Nesta aula, configuraremos o Redux Toolkit em uma aplicação React. Começaremos instalando as dependências necessárias (@reduxjs/toolkit e react-redux). Em seguida, configuraremos a store usando configureStore, que simplifica a configuração do Redux com boas práticas e middleware integrados. Criaremos slices de estado com createSlice, que combina reducers e actions em um único passo, simplificando a gestão do estado. Integraremos os slices na store e usaremos hooks do react-redux como useSelector e useDispatch para acessar e modificar o estado na interface do usuário. Também exploraremos createAsyncThunk para gerenciar operações assíncronas de forma eficaz. E, finalmente, discutiremos como o Redux Toolkit melhora a produtividade e a legibilidade do código ao fornecer uma API mais intuitiva e menos boilerplate.

SAIBA MAIS

O que é Redux Toolkit?

Redux Toolkit é uma biblioteca oficial desenvolvida com o objetivo de simplificar o uso do Redux na gestão de estados em aplicações JavaScript. Ele fornece uma série de ferramentas e funções que tornam a configuração e o uso do Redux mais fácil e eficiente, reduzindo significativamente a quantidade de código boilerplate necessário. Ele foi criado em resposta às críticas comuns sobre a complexidade e verbosidade do Redux clássico, oferecendo uma abordagem mais intuitiva e moderna para gerenciar o estado da aplicação.

Por que usar Redux Toolkit?

Uma das principais vantagens do Redux Toolkit é a **simplificação** da configuração da store por meio da função `configureStore`. Essa função não só configura a store, mas também aplica middlewares como `redux-thunk` e habilita as DevTools do Redux automaticamente, eliminando a necessidade de configuração manual adicional. Isso facilita a inicialização de novos projetos e a manutenção dos existentes.

Outra vantagem é a função `createSlice`, que combina a criação de **reducers e actions** em um único passo. Isso torna o processo mais direto e intuitivo, permitindo que as pessoas desenvolvedoras definam reducers e actions dentro de um único módulo, reduzindo a complexidade e a fragmentação do código.

Boas práticas

Imutabilidade: ele utiliza a biblioteca Immer para gerenciar a imutabilidade do estado. Isso significa que as atualizações do estado são garantidas como imutáveis sem que o(a) desenvolvedor(a) precise lidar diretamente com a complexidade dessa imutabilidade. Immer permite que quem desenvolve escreva um código mutável de forma segura, transformando-o automaticamente em atualizações imutáveis, o que simplifica o desenvolvimento e reduz a possibilidade de erros.

Middleware padrão: inclui middlewares padrão que ajudam a evitar erros comuns e a seguir as melhores práticas do Redux. Esses middlewares garantem que o fluxo de dados na aplicação seja gerenciado de maneira eficaz e segura,

proporcionando uma base sólida para a construção de aplicações escaláveis e robustas.

Produtividade

Menos boilerplate: uma das queixas mais comuns sobre o Redux clássico é a quantidade de código boilerplate necessário para configurar e utilizar o Redux. O Redux Toolkit reduz drasticamente essa quantidade de código, permitindo que os(as) desenvolvedores(as) se concentrem mais na lógica da aplicação e menos na configuração do Redux.

API intuitiva: o Redux Toolkit fornece uma API mais fácil de usar e entender, especialmente para iniciantes. As funções e ferramentas fornecidas são desenhadas para serem intuitivas, o que facilita o aprendizado e a implementação correta do Redux, mesmo para desenvolvedores com pouca experiência.

Integração assíncrona

A função `createAsyncThunk` facilita a criação e o gerenciamento de ações assíncronas, como chamadas de API. Ela simplifica o código necessário para lidar com operações assíncronas, melhorando a legibilidade e a organização do código. Essa função trata automaticamente os estados de `loading`, `success` e `error`, proporcionando uma forma estruturada e limpa de gerenciar side effects.

Quando usar Redux Toolkit?

Redux Toolkit é ideal para determinados projetos, como:

Para **aplicações complexas** onde múltiplos componentes precisam acessar e modificar o estado centralizado, o Redux Toolkit oferece uma maneira eficiente e organizada de gerenciar o estado da aplicação. Ele facilita a implementação de um estado global, permitindo que os componentes compartilhem dados e funcionalidades de maneira eficaz.

Aplicações que dependem de chamadas de API e outras operações assíncronas se beneficiam das ferramentas oferecidas pelo Redux Toolkit. A função `createAsyncThunk` simplifica o gerenciamento dessas operações, proporcionando uma abordagem clara e estruturada para lidar com side effects.

Para equipes que desejam manter um código mais limpo e fácil de manter, **buscando reduzir o boilerplate**, o Redux Toolkit é uma excelente escolha. Ao reduzir

a quantidade de código boilerplate necessário para configurar e utilizar o Redux, ele permite que as pessoas desenvolvedoras se concentrem na lógica de negócios da aplicação.

O Redux Toolkit transforma a maneira como os(as) desenvolvedores(as) utilizam o Redux, proporcionando simplicidade, boas práticas e produtividade. Com ferramentas como `configureStore`, `createSlice` e `createAsyncThunk`, ele torna o gerenciamento de estados mais acessível e eficiente, especialmente em projetos complexos que exigem um gerenciamento de estado centralizado e operações assíncronas. Ao reduzir o boilerplate e oferecer uma API intuitiva, o Redux Toolkit facilita a vida de quem desenvolve, promovendo a criação de aplicações escaláveis e robustas.

Como implementar

1. Instalação

Para começar a usar o Redux Toolkit, instale as dependências necessárias:

```
npm install @reduxjs/toolkit react-redux
```

2. Configuração da Store

Utilize `configureStore` para criar a store. Isso simplifica a configuração e aplica middlewares padrão:

```
import { configureStore } from '@reduxjs/toolkit';
import rootReducer from './slices';

const store = configureStore({
  reducer: rootReducer,
});

export default store;
```

`rootReducer`: Combina todos os slices criados, fornecendo uma estrutura centralizada para o estado da aplicação.

3. Criação de slices

Um slice é uma porção do estado da aplicação, que inclui o estado inicial, os reducers e as ações. Crie slices usando createSlice:

```
import { createSlice } from '@reduxjs/toolkit';

const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: state => { state.value += 1 },
    decrement: state => { state.value -= 1 },
  },
});

export const { increment, decrement } = counterSlice.actions;
export default counterSlice.reducer;
```

name: nome do slice.

initialState: estado inicial do slice.

reducers: funções que especificam como o estado deve mudar em resposta às ações.

4. Integração no componente

Utilize os hooks useSelector e useDispatch do react-redux para acessar e modificar o estado no componente:

```
import React from 'react';
import { useSelector, useDispatch } from 'react-redux';
import { increment, decrement } from './slices/counterSlice';

const Counter = () => {
  const count = useSelector(state => state.counter.value);
  const dispatch = useDispatch();

  return (
    <div>
      <span>{count}</span>
      <button onClick={() => dispatch(increment())}>Increment</button>
      <button onClick={() => dispatch(decrement())}>Decrement</button>
    </div>
  );
};
```

```
);
};

export default Counter;
```

useSelector: seleciona uma parte do estado da store.

useDispatch: despacha ações para modificar o estado.

5. Gerenciamento de operações assíncronas

Use createAsyncThunk para definir operações assíncronas, como chamadas API:

```
import { createAsyncThunk } from '@reduxjs/toolkit';

export const fetchData = createAsyncThunk(
  'data/fetchData',
  async () => {
    const response = await fetch('/api/data');
    return response.json();
  }
);
```

createAsyncThunk: facilita a criação de ações assíncronas que podem despachar automaticamente ações de sucesso e falha.

6. Adicionando Middleware e ferramentas

Redux Toolkit integra middlewares e ferramentas, como redux-thunk e redux-devtools-extension, por padrão:

redux-thunk: permite escrever criadores de ações que retornam uma função em vez de uma ação.

redux-devtools-extension: facilita a depuração do estado da aplicação.

Exemplo completo:

```
// store.js
import { configureStore } from '@reduxjs/toolkit';
import counterReducer from '../slices/counterSlice';

export default configureStore({
  reducer: {
    counter: counterReducer,
  },
});

// slices/counterSlice.js
import { createSlice } from '@reduxjs/toolkit';

export const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: (state) => { state.value += 1; },
    decrement: (state) => { state.value -= 1; },
  },
});

export const { increment, decrement } = counterSlice.actions;
export default counterSlice.reducer;

// App.js
import React from 'react';
import { Provider } from 'react-redux';
import store from './store';
import Counter from './Counter';

function App() {
  return (
    <Provider store={store}>
      <Counter />
    </Provider>
  );
}

export default App;

// Counter.js
import React from 'react';
import { useSelector, useDispatch } from 'react-redux';
import { increment, decrement } from '../slices/counterSlice';
```

```
const Counter = () => {
  const count = useSelector((state) => state.counter.value);
  const dispatch = useDispatch();

  return (
    <div>
      <span>{count}</span>
      <button onClick={() => dispatch(increment())}>Increment</button>
      <button onClick={() => dispatch(decrement())}>Decrement</button>
    </div>
  );
};

export default Counter;
```

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos o Redux Toolkit, entendendo sua importância e vantagens no gerenciamento de estado em aplicações React. Configuramos a store com `configureStore` e criamos slices de estado com `createSlice`. Integramos os slices na store e utilizamos hooks do `react-redux` para interagir com o estado na interface do usuário. Também aprendemos a gerenciar operações assíncronas com `createAsyncThunk`. Discutimos como o Redux Toolkit simplifica o uso do Redux, reduz o código boilerplate e melhora a produtividade e legibilidade do código.

REFERÊNCIAS

HAMIDREZA, A. How to use Redux in React Native. **Medium**. 2020. Disponível em: <https://hamidrdeveloper.medium.com/how-to-use-redux-in-react-native-c02058c8e8b4>. Acesso em: 14 ago. 2024.

IT SERVICES INDIA. How to use Redux in your React Native app. **Medium**. 2023. Disponível em: <https://medium.com/@itservicesindia/how-to-use-redux-in-your-react-native-app-99a18f382cef>. Acesso em: 14 ago. 2024.

REDUX. Getting Started. **Redux**. [s.d.]. Disponível em: <https://redux.js.org/introduction/getting-started>. Acesso em: 14 ago. 2024.

PALAVRAS-CHAVE

React native. Front-end. Redux-toolkit.

EXEMPLO

POS TECH